

Modelo Relacional de la Orden de Trabajo (OT) — AMS

Fecha: 2026-05-12
Owner: David Cabezas
Versión: 1.0
Audiencia: Carlos Iturra, Magdalena Ortega, Jorge Alquinta

1. Vista general del grafo



[illegible][illegible]

wo_ids[] wo_id ref (post-cierre)

[illegible]

...

La tabla `managed_work_orders` no usa joins SQL para sus relaciones internas — guarda los hijos como JSON arrays para tomar snapshot inmutable en cada estado.

Por qué JSON embebido en vez de tablas separadas:

- Snapshot inmutable de cada estado (operaciones al programar ≠ operaciones al cerrar)
- Menos joins SQL → vistas más rápidas en dashboards
- Schema flexible para campos custom por planta/cliente
- Trade-off: pierde joins/group-by clásicos. Se compensa con lookups en Python en endpoints específicos.

Tabla origen	Campo	Apunta a
`work_requests.source_capture_id`	FK	`field_captures.capture_id`

`managed_work_orders.absorbed_by_wo_id`	self-ref	`managed_work_orders.wo_id` (OT PM03 absorbe PM01/PM02)
---	----------	---

Pocos FKs explícitos a propósito — el resto son **lógicos** (apuntan por ID sin constraint DB) para tolerar:

- Borrado de captures sin romper la WR
- Borrado de WRs sin romper la OT (la OT puede ser standalone fast-track P1)
- Migración multi-plant sin cascadas

4. Relaciones lógicas (sin FK formal)

Origen	Campo	Apunta a	Para qué
`managed_work_orders.work_request_id`	string	`work_requests.request_id`	Trazabilidad WR → OT
`managed_work_orders.equipment_id`	string	`hierarchy_nodes.node_id` o `tag`	Equipo físico
`managed_work_orders.technical_location`	string	`hierarchy_nodes` (TL SAP)	Ubicación técnica SAP-PM
`managed_work_orders.contractor_crew_id`	string	`contractor_crews.crew_id`	Cuadrilla externa (nullable)
`managed_work_orders.assigned_workers[].worker_id`	string	`workforce.worker_id`	Técnicos asignados
`managed_work_orders.support_equipment[].tag`	string	`support_equipment.tag`	Grúas, andamios, etc.
`managed_work_orders.materials[].code`	string	`inventory_items.material_code`	Repuestos SAP
`managed_work_orders.materials[].reservation_code`	string	(sistema externo SAP)	Reserva BOM en SAP
`audit_log.entity_id` (donde `entity_type`='managed_work_order')	string	`wo_id`	Trazabilidad universal
`preparativos_ot.wo_id` (SF-662)	string indexed	`wo_id`	Tracking despacho bodega → patio
`backlog_items.wo_id`	string indexed	`wo_id`	Cola pre-programación
`weekly_programs.wo_ids[]` (JSON)	list	`wo_id`	Programa semanal
`rca.wo_id`	string	`wo_id`	RCA post-cierre
`improvement_actions.wo_id` (cuando aplica)	string nullable	`wo_id`	CAPA derivada de OT

5. Flujo de vida típico

...

1. Técnico abre app móvil → FieldCapture (capture_id)

↓

2. POST /capture/ crea WR (status=DRAFT, código AV-NNNNN)

↓

3. Supervisor /work-requests/{id}/validate {action: APPROVE}

→ WR.status = APROBADO

→ backlog_service.add_to_backlog() crea BacklogItem

↓

4. Planner crea OT: POST /managed-work-orders/from-wr

→ ManagedWorkOrder (wo_id, OT-2026-NNNNN, status=CREADO)

→ WR.status = LINKED

↓

5. Walk-through OT:

/release → /plan → /schedule (drag-drop Horarios) → /start

(cada transición = audit_log entry)

↓

6. Técnico ejecuta + notifica HH parcial (PUT /progress)

↓

7. Supervisor /close con signature PIN + gate_acks

→ pre-close gates validados:

- ALL_OPS_DONE (operaciones 100%)
- OPS_HH_NOTIFIED (HH real por op)
- HH_VARIANCE_OK (plan vs real ≤ 25%)
- SUPERVISOR_QA (firma manual)

→ status=CERRADO + closed_by_pin_hash + closure_audio_url

→ audit_log final

↓

8. Post-cierre (opcional):

- RCA si fue falla crítica
- improvement_actions si surge CAPA
- lessons_learned indexadas en RAG
- SF-661 función 7 (RCA hint IA — pendiente SP8)

...

6. Tablas universales (no específicas de OT)

Tabla	Propósito	Conecta a OT vía
`audit_log`	Trazabilidad inmutable de todo el sistema	`entity_type='managed_work_order' + entity_id=wo_id`

`field_captures`	Capturas de campo (foto + audio + GPS)	Vía WR via `source_capture_id`
`hierarchy_nodes`	Estructura jerárquica del activo (Plant→Area→System→Equipment)	`equipment_id` lógico
`workforce`	Personal técnico	`assigned_workers[].worker_id` lógico
`inventory_items`	Catálogo materiales SAP	`materials[].code` lógico
`support_equipment`	Grúas + andamios + herramientas	`support_equipment[].tag` lógico

7. Decisiones de diseño relevantes

7.1 Por qué JSON embebido en vez de tablas separadas

- **Snapshot inmutable** por estado: las operaciones al programar pueden diferir de las al cerrar.
- **Trazabilidad temporal**: `description_history` mantiene cada edición con autor + timestamp.
- **Schema flexible**: campos custom por planta/cliente sin migraciones.
- **Performance dashboards**: menos joins para vistas tipo Executive.

7.2 Por qué `work_request_id` NO tiene FK constraint

- **Standalone fast-track**: OTs P1/P2 pueden crearse sin WR previo (incidente imprevisto).
- **Robustez ante migraciones**: borrado de WR no rompe OTs ya producidas.
- **Compatibilidad SAP**: en SAP el aviso y la orden son entidades semi-independientes.

7.3 Concurrency control

- Campo `version` en `managed_work_orders` (SF-562) implementa optimistic locking.
- Frontend manda If-Match: `<version>` en PUT.
- Backend retorna 409 Conflict si la versión cambió.
- Excepciones: `support_equipment` y `materials.reservation_code` bypass del lock (metadata accesoria — Jorge 2026-04-28).

7.4 Soft-delete vs hard-delete

- OTs NO se borran físicamente. Estados terminales: `CERRADO` , `CANCELADO` .
- Audit log: append-only (SF-660 política inmutabilidad).
- Solo el script `scripts/purge_audit_log_5y.py --confirm` puede eliminar audit > 5 años.

7.5 Soporte multi-plant

- `plant_id` en `managed_work_orders` (string, no FK formal contra `plants`).
- IDOR-scoping en todos los endpoints: si el usuario no es admin/manager, su `plant_id` JWT gatea acceso.
- Tests `test_security_*` cubren los escenarios cross-plant.

8. Datos curiosos de cardinalidad

Relación	Cardinalidad típica
FieldCapture → WR	1:1 (cada captura genera 1 WR)
WR → OT	1:0..1 (cada WR genera máximo 1 OT — o ninguna si rechazada)
OT → operations	1:N (típico 5-15 operaciones por OT)
OT → materials	1:N (0-50 materiales según tipo)
OT → assigned_workers	1:N (típico 1-5 técnicos por OT)
OT → audit_log entries	1:N (típico 15-40 entries lifecycle completo)
OT → preparativos	1:N (típico 0-10 items por OT crítica)
OT PM03 → OTs absorbidas	1:N self-ref

9. Tipos de OT y su impacto en el flujo

`wo_type`	Origen	Skip planning?	Pre-close gates
PM01 — Correctivo	WR de falla	No	Todas las gates
PM02 — Preventivo	Plan matriz	No	Todas
PM03 — Mejora	Improvement action	No	Todas + absorbe PM01/PM02
Fast-track P1	Imprevisto urgente	Sí — va directo a PROGRAMADO	Todas + supervisor_qa obligatorio
Fast-track P2	Programa en ejecución	Sí	Todas

10. Referencias cruzadas

- **Políticas:** [docs/AUDIT_LOG_POLICY.pdf](AUDIT_LOG_POLICY.pdf), [docs/OT_CALENDAR_SYNC.pdf](OT_CALENDAR_SYNC.pdf)
- **Modelo Python:** `api/database/models.py` líneas 456 (WR), 534 (OT)
- **Service layer:** `api/services/managed_wo_service.py` — TRANSITIONS, `compute_close_gates`, `close_wo`
- **Tests:** `tests/test_api/test_managed_wo.py` , `tests/test_api/test_ot_calendar_sync.py`
- **Diagrama OT-Calendar sync:** matriz 9 campos + 5 reglas conflictos en `docs/OT_CALENDAR_SYNC.pdf`

Generado en commit: `cd83670` · `bdfbcc3` (post fix-tests + SF-661 v0.2)